Capstone Exam: ModelServe

MLOps with Cloud Season 2

Poridhi.io

Table of Contents

MLOps with Cloud — Season 2

Capstone Exam: ModelServe

Episodes Covered: - Episode 1 — MLOps & Cloud Foundations - Episode 2 — AWS Cloud Infrastructure for ML - Episode 3 — Docker & Containerization for MLOps

Format: 10 lab sessions × 6 hours · GitHub-persisted between sessions **Submission:** Public GitHub repository + live deployment demo with a Teaching Assistant **Instructor:** Tahnik Ahmed

What you will build: A production-grade ML serving platform — MLflow registry, Feast feature store, FastAPI inference service, Prometheus + Grafana observability — all containerized, deployed across infrastructure you design, provisioned via Pulumi, and automated through a GitHub Actions CI/CD pipeline.

What you will not optimize for: Kaggle leaderboard ranking. Train a reasonable baseline, register it in MLflow, and invest your time in the infrastructure, observability, automation, and documentation around it.

What makes this exam different: There is no prescribed architecture. You decide what runs where — entirely on a Poridhi VM, entirely on AWS EC2, or split across both. You design the deployment topology, defend it in writing, draw the architecture diagrams, and explain it live. The documentation you write is as important as the code you ship.

Stack at a glance: FastAPI · MLflow · Feast · Redis · PostgreSQL · Prometheus · Grafana · Docker · Pulumi · AWS (EC2, ECR, S3, VPC, IAM) · GitHub Actions

---

Table of Contents

---

1. The Big Picture

1.1 What Is ModelServe?

ModelServe is a production-grade ML serving platform that students will design, build, deploy, and document as the capstone of the first three episodes of *MLOps with Cloud Season 2*. The core exercise is wrapping a trained ML model in the production infrastructure that makes it actually serveable: experiment tracking, a feature store, a containerized inference API, full observability, cloud infrastructure provisioned as code, and an automated CI/CD pipeline.

Students will train a model on a Kaggle dataset (announced separately), but model quality is deliberately de-emphasized — a solid baseline is all that's needed. The bulk of the effort goes into everything around the model, and into the **engineering documentation** that explains *why* the system is built the way it is.

1.2 Why This Capstone?

Each module across Episodes 1–3 taught one isolated skill. ModelServe forces students to combine every skill into a coherent system and make real engineering decisions under realistic constraints:

- Where does each service live — on the Poridhi VM, on AWS EC2, or split across both? Why?
- What backs MLflow's tracking store and artifact store, and how does that choice affect your deployment?
- What features belong in Redis, what stays in S3, and when is the cache stale?
- What is the right Docker base image for a Python ML service, and how does multi-stage building actually pay off?
- What does the security group need to allow — and more importantly, what should it deny?
- Should the CI/CD pipeline destroy and recreate infrastructure on every push, or update it incrementally?
- When the system is on fire at 3 AM, which Grafana panel tells you *which thing* is on fire?

These are the questions ML platform engineers answer every day. ModelServe provides a controlled environment to answer them — and then **document the answers** in a way that another engineer could understand and reproduce.

1.3 The ML Task

Students will train their own model as part of this capstone. After this document is published, the course will announce a **Kaggle notebook or competition link** that defines the dataset and prediction task. The specific dataset and task will be disclosed at that time — do not begin model training until the link is published.

Regardless of the task, the following constraints apply:

- The model must be a `scikit-learn` compatible estimator (any algorithm from sklearn, XGBoost, or LightGBM is acceptable). Deep learning frameworks are out of scope for this capstone.
- The trained model must be serializable as a `.pkl` file small enough to fit comfortably in the MLflow artifact store (under 500 MB).
- Students must produce a `train.py` script that trains the model and registers it in MLflow with appropriate metrics, parameters, and the model artifact. This script must be reproducible — another person running `python train.py` with the same data should get a functionally equivalent model.
- Students must produce a `features.parquet` file suitable for Feast ingestion, derived from the Kaggle dataset's feature columns.

- Students must produce a `sample_request.json` file containing a valid prediction request payload for testing their deployed API.

The model quality is not the focus. A simple baseline that scores reasonably is perfectly acceptable. Students who spend their sessions optimizing model accuracy at the expense of the MLOps infrastructure will be marked down, not up. The grading rubric weights infrastructure, observability, documentation, and CI/CD far more heavily than model performance.

1.4 System Components

The finished system must include the following components. **How and where you deploy them is your decision** — see Section 1.5.

Component	Technology	Responsibility
Inference API	FastAPI + Uvicorn/Gunicorn	Serves `/predict`, `/health`, `/metrics`; loads model from MLflow Registry
Feature Store	Feast + Redis (online) + file/S3 (offline)	Provides feature lookups for incoming prediction requests
Experiment Tracking	MLflow Tracking Server + Postgres + file/S3 artifacts	Stores model versions, metrics, and artifacts
Metrics Collection	Prometheus	Scrapes `/metrics` endpoints from services
Dashboards & Alerts	Grafana	Provisioned dashboards and alert rules for the full stack
Infrastructure as Code	Pulumi (Python)	Provisions AWS resources used by the system
CI/CD	GitHub Actions	Automates testing, building, and deployment on push to main
Containerization	Docker + Docker Compose	Packages all services into reproducible containers

1.5 Deployment Topology — Your Decision

There is no prescribed architecture for this exam. You choose the deployment topology, and you defend it in your documentation. Here are three examples of valid approaches — you are not limited to these:

Option A — Everything on a single AWS EC2 instance. All containers run on one EC2 box provisioned by Pulumi. Simple, easy to reason about, mirrors the Pulumi/CI/CD lab. Trade-off: single point of failure, resource contention between services.

Option B — Everything on the Poridhi VM. Services run locally on the Poridhi VM via Docker Compose. AWS is used only for S3 (MLflow artifacts, Feast offline store) and ECR (image registry). Trade-off: no cloud compute, but simpler networking; the Poridhi VM doesn't persist between sessions, so you need a fast bootstrap.

Option C — Hybrid split. MLflow server, Postgres, and the monitoring stack run on the Poridhi VM (they don't need to face the internet). The FastAPI inference service runs on an AWS EC2 instance behind an Elastic IP, pulling the model from MLflow across the network and features from a Redis that could be on either side. Trade-off: more realistic multi-host deployment, but more complex networking and debugging.

You may also design a topology not listed here. The only hard requirements are:

1. The FastAPI inference service must be reachable via a stable URL or IP that the TA can hit during the demo.
2. Pulumi must provision at least some AWS resources (you cannot skip IaC entirely).
3. GitHub Actions must automate at least the test, build, and deploy steps.
4. Your documentation must include an architecture diagram that shows exactly what runs where and how the components communicate.

The grading rubric does not favor any particular topology. A simple, well-documented single-node deployment scores the same as a complex multi-host split — provided the documentation explains the reasoning and the trade-offs are acknowledged.

1.6 Key User Flows

The following four flows define the system and are used as acceptance criteria during grading:

1. A user sends `POST /predict` with an entity ID → the FastAPI service fetches features for that entity from Feast (Redis online store) → loads the latest production model from the MLflow Registry → returns a prediction with model version and timestamp.

2. A user sends `GET /predict/<entity_id>?explain=true` → the FastAPI service returns the prediction *plus* the feature values that were used, enabling debugging.

3. A TA sends a burst of requests using a load testing tool → the Grafana dashboard shows real-time prediction latency (p50, p95, p99), request rate, and error rate updating live → a Prometheus alert fires when p95 latency exceeds the student's configured threshold.

4. A developer pushes a change to `main` → GitHub Actions runs tests → infrastructure is provisioned/updated → Docker images are built and pushed → the deployment target pulls new images and restarts → within a reasonable time the new version is live and serving traffic.

1.7 What Students Submit

A single public GitHub repository containing:

- One `docker-compose.yml` at the root that starts the local development stack with `docker compose up`
- An `infrastructure/` directory with Pulumi code that provisions AWS resources
- A working `.github/workflows/deploy.yml` that runs the full CI/CD pipeline
- Multi-stage `Dockerfiles` for each custom service

- A `monitoring/` directory with Prometheus config, alert rules, and Grafana provisioning
- A `docs/` directory containing the **Engineering Documentation** (see Section 5.2) — this is a major deliverable, not an afterthought
- A `README.md` with setup instructions and environment variable documentation
- All tests passing in CI on the latest commit to `main`

Students will demonstrate the deployed system live to a TA and answer questions about their architectural choices.

---

2. Sandbox & Environment

2.1 What Poridhi Provides

Each session, students receive:

- **A fresh Poridhi VM** with Docker, Python 3.10+, AWS CLI, Pulumi, Git, and Node.js pre-installed. This can serve as both a workstation and a deployment target.
- **A fresh AWS sandbox** with credentials valid for the duration of the session (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, region `ap-southeast-1`). Resources created in the sandbox are automatically destroyed when the session ends.
- **A starter GitHub repository template** (forked once at the start of Session 1) containing a skeleton directory structure and placeholder configuration files. The Kaggle dataset link will be announced separately.

2.2 What Students Are Responsible For

- **Pushing all work to GitHub at the end of every session.** Anything not pushed is lost when the sandbox terminates. This is the single most important operational rule of the capstone.
- **Tearing down AWS resources at the end of every session.** Run `pulumi destroy --yes` before the session ends. The sandbox will clean up automatically, but explicit teardown is good hygiene and avoids dangling resources that interfere with the next session.
- **Configuring GitHub Secrets** in their forked repository so the CI/CD pipeline can authenticate to AWS and connect to deployment targets.
- **Pulling the latest code at the start of every session** with `git clone` or `git pull`, then re-bootstrapping the local environment.

2.3 Resource Guidelines

Students may use AWS resources within the sandbox's capabilities. The following are **guidelines**, not hard limits — if your architecture requires a different resource mix, document the reasoning in your engineering documentation.

Typical resource envelope:

- 1–2 EC2 instances (t3.small or t2.micro)

- 1–2 ECR repositories
- 1 S3 bucket
- 1 VPC with public subnet(s)
- 0–1 Elastic IPs

Out of scope for this capstone: RDS, EKS, ALB, NAT Gateway, Lambda, SageMaker, or any managed ML/container service. These are covered in later episodes. If you use them, you must justify why in your documentation, and the TA will probe this decision during the demo.

---

3. Session-by-Session Milestones

The exam is structured as 10 sessions of 6 hours each. Each session has a suggested milestone, but the milestones are **soft gates** — a student who falls behind in one session can catch up in the next, and a student who finishes early can move forward. The grading happens at the end, against the final state of the repository, the documentation, and the live demo.

> **The Golden Rule:** Every session ends with `git push`. Every session begins with `git pull` followed by `docker compose up` to verify the previous session's work still runs locally. If your local stack doesn't come up cleanly at the start of a session, that is the first thing you fix.

Sessions 1–2 — Foundation: Model Training & Local Stack

Goal by end of Session 2: A working local Docker Compose stack with MLflow, FastAPI, Feast, and Postgres/Redis, serving predictions from your trained model.

What students build:

In Session 1, students fork the starter template, set up their Poridhi VM, and download the Kaggle dataset. They write `train.py` — a script that loads the data, trains a baseline sklearn-compatible model, logs metrics and parameters to MLflow, and registers the model in the MLflow Model Registry with stage `Production`. They also produce `features.parquet` from the dataset's feature columns and `sample_request.json` with a valid test payload.

In Session 2, students build the FastAPI inference service. The service exposes `GET /health`, `POST /predict`, and `GET /metrics`. The `/predict` endpoint loads the model from the MLflow Registry on startup, fetches features from the Feast online store (Redis), and returns predictions. Students stand up a local Feast repository and materialize features into Redis.

Acceptance criteria:

- `docker compose up` from the repo root starts MLflow, Postgres, Redis, and the FastAPI service without error.
- At least one model exists in the MLflow Registry with logged metrics, parameters, and a version in the `Production` stage.

- `train.py` is reproducible — running it again registers a new model version with comparable metrics.
- `curl -X POST http://localhost:8000/predict -d @sample_request.json` returns a valid prediction response.
- `curl http://localhost:8000/health` returns 200.

Sessions 3–4 — Containerization & Observability

Goal by end of Session 4: Production-grade Dockerfiles, Prometheus scraping, Grafana with provisioned dashboards, and at least three meaningful alert rules.

What students build:

In Session 3, students write multi-stage Dockerfiles. The final runtime image must run as a non-root user, use a production WSGI/ASGI server, and include a `HEALTHCHECK` directive. The FastAPI image must be under 800 MB.

In Session 4, students add Prometheus and Grafana to Docker Compose. Prometheus scrapes the FastAPI `/metrics` endpoint. Grafana provisioning files automatically create the datasource and load a dashboard from JSON — no manual configuration after startup. The dashboard must show at minimum: prediction latency p50/p95/p99, request rate, error rate, model version, and Feast hit/miss ratio. Students write at least three Prometheus alert rules covering high latency, elevated error rate, and service-down scenarios.

Acceptance criteria:

- The FastAPI image is under 800 MB and uses a multi-stage build.
- The container runs as a non-root user.
- Prometheus shows the FastAPI service as `up`.
- The Grafana dashboard renders automatically after `docker compose up`.
- At least three alert rules are defined and visible in the Prometheus UI.

Sessions 5–7 — Cloud Infrastructure with Pulumi & Deployment

Goal by end of Session 7: The system deployed and accessible on your chosen infrastructure, provisioned by Pulumi.

What students build:

This is the most substantial block. Students write Pulumi code (Python) to provision the AWS resources their chosen topology requires. At minimum this includes some combination of VPC, subnet, security group, EC2 instance, S3 bucket, ECR repository, and IAM role — the exact mix depends on the student's architecture.

Students deploy the full stack to their chosen target(s) and verify end-to-end functionality: predictions work, the Grafana dashboard shows live metrics, and MLflow serves the model.

This is also when students begin writing their Engineering Documentation (see Section 5.2). The architecture diagram and initial ADRs should be drafted by end of Session 7, while the deployment topology decisions are fresh.

Acceptance criteria:

- `pulumi up --yes` provisions all required AWS resources without errors.
- The FastAPI inference service is accessible via a stable IP or URL.
- `curl -X POST http://<target>:8000/predict -d @sample_request.json` returns a valid prediction.
- Grafana at `http://<target>:3000` renders the monitoring dashboard.
- `pulumi destroy --yes` cleanly removes all provisioned resources.

Sessions 8–9 — GitHub Actions CI/CD

Goal by end of Session 9: A `git push` to `main` triggers a complete test, build, and deploy cycle.

What students build:

Students write a GitHub Actions workflow with at least three jobs:

1. **`test`** — runs on every push and pull request. Runs `pytest` against the FastAPI service.
2. **`build-and-push`** — runs on pushes to `main` only, after `test` passes. Builds Docker images and pushes to ECR.
3. **`deploy`** — runs after `build-and-push`. Deploys the new images to the target infrastructure and verifies the health endpoint.

Students may add additional jobs (e.g., a separate `infrastructure` job for `pulumi up`, a linting job, a security scanning job). The exact pipeline structure is the student's design choice and must be documented.

Students must make a deliberate choice between **destroy-and-recreate** and **incremental update** for infrastructure management, and document the choice in an ADR.

Acceptance criteria:

- Pushing to `main` triggers the workflow and all jobs pass.
- After a successful run, the deployed service returns 200 on `/health`.
- The pipeline can be re-run end-to-end without manual intervention between runs.

Session 10 — Documentation, Hardening, and Demo Rehearsal

Goal by end of Session 10: The repository is in its final state, the Engineering Documentation is complete, and the student has rehearsed the demo.

What students do:

- Complete the Engineering Documentation (Section 5.2) — this is not optional and carries significant weight.
- Polish the Grafana dashboard and verify all alerts work.
- Run their own end-to-end validation: destroy infrastructure, push to `main`, watch the pipeline run, verify the deployed system works.
- Rehearse the demo. Read Section 6 and Section 7 carefully.

- Optionally implement bonus features (Section 5.4).

The session should end with a clean `git push` and a clean `pulumi destroy`.

---

4. Component Contracts

This section specifies the contracts each component must satisfy. Students have full flexibility in how and where they deploy, but must meet these contracts.

4.1 FastAPI Inference Service

Endpoints:

Method	Path	Description
GET	`/health`	Returns `{"status": "healthy", "model_version": "<version>"}`
POST	`/predict`	Accepts `{"entity_id": <int>}`. Returns `{"prediction": <int>, "probability": <float>, "model_version": "<version>", "timestamp": "<iso8601>"}`
GET	`/predict/<entity_id>?explain=true`	Returns prediction plus the feature values used
GET	`/metrics`	Prometheus metrics. Must expose: `prediction_requests_total`, `prediction_duration_seconds`, `prediction_errors_total`, `model_version_info`

Behavioral requirements:

- The model must be loaded from the MLflow Registry on application startup, not on each request.
- Feature lookups must go through Feast's online store API (`feature_store.get_online_features(...)`), not by querying Redis directly.
- Prediction errors must return structured JSON with appropriate HTTP status codes and must increment `prediction_errors_total`.

4.2 MLflow Tracking Server

- Backend store: PostgreSQL.
- Artifact store: local directory for local development; S3 in production (student decides the specifics).
- At least one model registered with logged metrics, parameters, and a version in the `Production` stage.

4.3 Feast Feature Store

- Online store: Redis.
- Offline store: local Parquet files or S3 (student decides based on topology).

- Feature definitions written by the student. Materialized via `feast materialize-incremental`.
- The FastAPI service must read features through the Feast SDK.

4.4 Prometheus & Grafana

- At least three alert rules covering: high latency, high error rate, service down.
- Grafana datasource and dashboard provisioned via configuration files — no manual UI setup.
- Dashboard must show at minimum: latency p50/p95/p99, request rate, error rate, model version, Feast hit/miss ratio.

4.5 Pulumi Infrastructure

- Must be Python (not TypeScript or Go).
- Must provision at least some real AWS resources (the exact set depends on the student's topology).
- All resources must be tagged with `Project: modelserve`.
- Must export relevant stack outputs (IPs, repo URLs, bucket names) for downstream use by CI/CD.
- `pulumi destroy` must cleanly remove all resources.

4.6 GitHub Actions Workflow

- Must trigger on push to `main` (and optionally on pull requests for the test job).
- Must include at minimum: test, build-and-push, and deploy jobs.
- Must use GitHub Secrets for all credentials — no hardcoded secrets.

---

5. Final Submission Requirements

5.1 Repository Structure

The following is a recommended structure. Students may deviate if they document the reasoning.

```
modelserve/
├── app/
│   ├── main.py                    # FastAPI application
│   ├── model_loader.py            # MLflow model loading logic
│   ├── feature_client.py          # Feast online lookup wrapper
│   ├── metrics.py                 # Prometheus metric definitions
│   └── tests/
│       └── test_predict.py
├── training/
│   ├── train.py                   # Model training + MLflow registratio
n
│   ├── features.parquet           # Feast-compatible feature file
│   └── sample_request.json        # Valid test prediction payload
├── feast_repo/
│   ├── feature_definitions.py
│   └── feature_store.yaml
├── infrastructure/
```

```
│   ├── __main__.py                 # Pulumi program
│   ├── Pulumi.yaml
│   └── requirements.txt
├── monitoring/
│   ├── prometheus/
│   │   ├── prometheus.yml
│   │   └── alerts.yml
│   └── grafana/
│       ├── provisioning/
│       │   ├── datasources/prometheus.yml
│       │   └── dashboards/dashboard.yml
│       └── dashboards/
│           └── modelserve-overview.json
├── docs/
│   ├── ARCHITECTURE.md             # Full engineering documentation
│   └── diagrams/                   # Architecture diagrams (images or so
urce)
├── .github/
│   └── workflows/
│       └── deploy.yml
├── docker-compose.yml
├── Dockerfile
├── .env.example
├── .gitignore
└── README.md
```

5.2 Engineering Documentation (Major Deliverable)

The `docs/ARCHITECTURE.md` file is **one of the most heavily weighted deliverables** in this capstone. It is not a README appendix — it is a standalone engineering document that another engineer could read and fully understand the system without looking at the code. Poor or missing documentation will result in significant mark deductions even if the code works.

The document must contain all of the following:

5.2.1 System Overview — A 2–3 paragraph description of what the system does, who it serves, and the key design philosophy (e.g., simplicity over availability, cost-efficiency over performance, etc.).

5.2.2 Architecture Diagram(s) — At least one clear diagram showing: - Every component in the system (FastAPI, MLflow, Feast, Redis, Postgres, Prometheus, Grafana) - Where each component runs (Poridhi VM, AWS EC2, or both) - How components communicate (ports, protocols, network boundaries) - External dependencies (S3, ECR, GitHub Actions)

The diagram can be created with any tool: Excalidraw, draw.io, Mermaid, ASCII art, or even a hand-drawn diagram photographed and committed. **Clarity matters more than polish.** A messy but accurate diagram scores higher than a pretty but wrong one. Include the diagram as an image file in `docs/diagrams/` and reference it in the markdown.

If the system has distinct local-development and production topologies, include a diagram for each.

5.2.3 Architecture Decision Records (ADRs) — At least **five** ADRs in the following format:

```
### ADR-N: [Title]

**Context:** [What situation or problem prompted this decision?]

**Decision:** [What did you decide?]

**Rationale:** [Why this choice over the alternatives?]

**Trade-offs:** [What did you give up? What risks does this introduce?]
```

The five ADRs must cover at least the following topics (students may add more):

1. **Deployment topology** — Why you chose to deploy where you did (single-node, hybrid, etc.)
2. **CI/CD strategy** — Destroy-and-recreate vs. incremental update, and why
3. **Data architecture** — Why Postgres for MLflow backend, why Redis for Feast online store, why S3 for artifacts
4. **Containerization** — Base image choice, multi-stage build strategy, image size trade-offs
5. **Monitoring design** — Why these alert thresholds, what the dashboard is optimized to show, what's missing

5.2.4 CI/CD Pipeline Documentation — A description of the GitHub Actions workflow: what each job does, what triggers it, what secrets it needs, how it handles failures, and the expected end-to-end deploy time.

5.2.5 Runbook — A concise operations guide covering: - How to bootstrap the system from a fresh clone (step by step, including secrets setup) - How to deploy a new model version without restarting the whole stack - How to diagnose and recover from common failures (service crash, S3 permission loss, Pulumi state corruption) - How to tear down everything cleanly

5.2.6 Known Limitations — An honest list of what the system does *not* handle well, what you would improve with more time, and what would need to change for a real production deployment.

5.3 README.md

The README is a quick-start guide, not the engineering document. It must contain:

- Project description (2–3 sentences)
- Prerequisites (Docker, Python, AWS CLI, Pulumi)
- Quick setup instructions for local development
- A table of all REST endpoints
- The list of all environment variables with descriptions (matching `.env.example`)
- A "GitHub Secrets" section listing every secret the workflow needs (without values)
- A link to the full Engineering Documentation at `docs/ARCHITECTURE.md`

5.4 Grading Rubric

Component	Criteria	Max Marks
Local stack	Compose stack starts cleanly, all components healthy, model serves predictions locally	8
Model training	`train.py` is reproducible, logs to MLflow, registers model with metrics/params	7
Containerization	Multi-stage Dockerfile, image under 800 MB, non-root user, healthcheck	8
MLflow & Feast	Model registered correctly, features materialized, FastAPI uses both via SDK	8
Observability	Prometheus scraping, Grafana provisioned dashboard, three working alerts	10
Pulumi infrastructure	Resources provisioned correctly, tagged, clean up/destroy cycle	10
GitHub Actions CI/CD	Pipeline runs end-to-end on `git push`, all jobs pass	10
Engineering Documentation	**Architecture diagram(s), five ADRs, CI/CD docs, runbook, known limitations**	**19**
Live demo & Q&A	Student explains choices, debugs live, answers questions confidently	10
Total		**100**
Bonus	See below	+10

Bonus features (up to +10 marks):

Feature	Marks	Notes
Trivy scan in CI that fails on critical vulnerabilities	+2	Must actually catch a real vulnerability
Second model registered with A/B traffic splitting	+3	Show traffic split in Grafana
`/rollback` endpoint that switches to previous model version	+2	Demo live during the exam
Blue/green deployment in the CI pipeline	+3	Two instances with traffic swap
Custom Prometheus exporter for Feast stats	+2	Beyond basic hit ratio
Pulumi `preview` step that comments diff on PRs	+2	Advanced GitHub Actions usage
Test coverage above 80%	+1	Must be meaningful tests

5.5 Submission Checklist

Before the demo session, verify:

- The repository is public on GitHub

- `docker compose up` from a fresh clone reaches a healthy state in under 15 minutes
- The latest commit on `main` shows a green checkmark on GitHub Actions
- `pulumi destroy` was run at the end of the last session
- `docs/ARCHITECTURE.md` is complete with diagrams, five ADRs, runbook, and known limitations
- The README links to the engineering documentation and lists all endpoints and environment variables
- You have rehearsed the demo at least once
- You have read Section 7 of this document and prepared answers for the sample questions

---

6. The Live Demo Session

Each student schedules a separate 60-minute demo session with a TA after Session 10. The demo is a major grading component, and it cannot be made up — students who do not appear or who cannot demonstrate a working system will lose those marks.

6.1 Demo Structure (60 minutes)

Minutes 0–10: Cold start. The student starts a fresh Poridhi VM and a fresh AWS sandbox. They configure credentials, then trigger their CI/CD pipeline (push to `main` or manual trigger) to deploy from scratch. The TA observes the pipeline running.

Minutes 10–25: Architecture walkthrough. While the pipeline runs, the student walks the TA through their Engineering Documentation — the architecture diagram, the key ADRs, and the deployment topology. The TA will ask follow-up questions probing the student's understanding. Having the documentation open and clear is a significant advantage here.

Minutes 25–40: Live system demo. Once deployment is complete, the student demonstrates the running system: hits `/health`, makes predictions, opens Grafana, shows traffic flowing. The TA will generate load in the background and ask the student to identify the pattern in the dashboard.

Minutes 40–55: Stress testing and failure injection. The TA asks the student to perform specific tasks live — see Section 7 for examples of the kinds of tasks that may be requested. This tests both technical competence and the ability to debug under pressure.

Minutes 55–60: Wrap-up Q&A. Final questions and teardown with `pulumi destroy`.

6.2 What the TA Is Evaluating

- **Understanding over execution.** A student who explains a failed deployment clearly scores higher than one whose system works but who can't explain why.
- **Live debugging skill.** Things will break during the demo. Calm, systematic debugging with clear narration is rewarded.
- **Consistency between documentation and reality.** If the ADR says one thing but the code does another, the TA will notice.

- **Intellectual honesty.** "I tried X and it didn't work, so I went with Y instead" is a stronger answer than pretending everything went according to plan.

---

7. Exam Preparation Guide — Sample Questions & Tasks

This section gives you a sense of the kinds of questions the TA may ask and the tasks they may request during the live demo. **The actual demo questions will be drawn from a pool similar to these, but will not be identical.** Preparing thoughtful answers to these samples is the best way to prepare for the exam.

7.1 Sample Validation Checks

During the demo, the TA will verify that the deployed system passes basic checks like these:

1. `curl http://<target>:8000/health` returns 200 with a `model_version` field
2. `curl -X POST http://<target>:8000/predict -d @sample_request.json` returns 200 with valid JSON
3. `curl http://<target>:8000/metrics` returns Prometheus-format text containing `prediction_requests_total`
4. The Grafana dashboard is accessible and shows live data
5. Prometheus shows at least three configured alert rules
6. The MLflow UI is accessible and shows registered models
7. After a burst of requests, the Prometheus query `rate(prediction_requests_total[1m])` returns a non-zero value
8. AWS resources are tagged with `Project: modelserve`

7.2 Sample Questions the TA May Ask

Expect to be asked 3–5 questions like these. You should be able to answer any of them:

- **Walk me through `train.py`.** What metrics did you log, and why those? How would you decide this model is good enough to deploy?
- **Why did you choose this deployment topology?** Walk me through the architecture diagram. What would you change if you had two more weeks?
- **Why did you choose destroy-and-recreate (or incremental update) for your CI/CD pipeline?** When would the other choice be better?
- **Walk me through what happens between `git push` and the model serving its first request from the new version.** Every step, every job, every container restart.
- **Your FastAPI service loads the model on startup.** What's the trade-off versus loading it per-request? How would you handle a model that's too big to fit in memory?
- **Why is the model pulled from MLflow Registry instead of baked into the Docker image?** What are the trade-offs of each approach?
- **Why is Feast in this stack at all?** What would break if you replaced it with a direct Redis client? When would you *not* use a feature store?

- **Walk me through the security group.** Which ports are open and to whom? What would you tighten in a real production deployment?
- **What's in the user-data script for the EC2 instance?** What happens if it fails halfway through?
- **Show me the Prometheus alert rules.** Why did you pick those thresholds? How would you choose them for a system handling real traffic?
- **Your S3 bucket holds both MLflow artifacts and Feast offline data.** What would go wrong if the IAM role lost S3 read permissions? How would you detect it before a customer does?
- **If the deployment target died right now, walk me through recovery.** How long would it take? What data would be lost?

7.3 Sample Tasks the TA May Request Live

During the "stress testing and failure injection" portion of the demo, the TA may ask you to do things like:

- **Kill the FastAPI container** and show that the Prometheus alert fires within the expected time.
- **Trigger a deliberate model loading error** and show how it surfaces in the `/metrics` endpoint and the Grafana dashboard.
- **Roll back to a previous commit** and redeploy. Verify the old version is serving.
- **Add a new field** to the prediction response (e.g., `"confidence_interval": [low, high]`), push the change, and show it deploys through the pipeline.
- **Explain what would happen** if someone pushed a commit that breaks the test suite. At what stage does the pipeline stop?
- **Show the TA a specific metric** in Grafana — e.g., "Show me the p99 latency for the last 5 minutes" or "Show me the total number of predictions since the last deploy."

7.4 Common Pitfalls to Avoid

These are patterns the TAs have seen in past cohorts that result in mark deductions:

- **The pipeline passes but the deployed service errors on every request.** This means the tests are mocking too aggressively — write at least one integration test that actually loads the model and makes a prediction.
- **Grafana dashboard exists but shows no data.** Usually the datasource provisioning works but the dashboard JSON references the wrong datasource UID. Test this end-to-end before the demo.
- **`pulumi destroy` fails because ECR has images.** Set `force_delete=True` on the repository or clean up images before destroying.
- **The documentation says one thing, the code does another.** TAs check this. If your ADR says "I chose Redis for feature caching because of sub-millisecond latency" but your code doesn't use Redis at all, that's worse than having no ADR.
- **The architecture diagram is missing or is a vague box-and-arrow sketch with no labels.** The diagram must show specific technologies, ports, and deployment locations. A well-labeled ASCII diagram is better than a pretty but vague one.

- **The student cannot explain their own code.** If you used an AI tool or copied from the lab, understand what you copied and why. The Q&A is designed to distinguish understanding from copying.

MLOps with Cloud Season 2 — Capstone: ModelServe | Poridhi.io